

Creating Servlets and Configuring using web.xml

1. Creating a Servlet

Steps to create a servlet:

1. Import required packages
2. Extend HttpServlet class
3. Override doGet() or doPost() method
4. Write logic to handle request and response

Example Servlet Code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Hello, Servlet!</h2>");
        out.println("</body></html>");
    }
}
```

2. Compiling the Servlet

- Compile using servlet API (jar file required)

```
javac HelloServlet.java
```

3. Configuring Servlet in web.xml

web.xml is called the **deployment descriptor**. It is used to configure servlets.

Example web.xml:

```
<web-app>

    <servlet>
        <servlet-name>HelloServlet</servlet-name>
        <servlet-class>HelloServlet</servlet-class>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
  <servlet-name>HelloServlet</servlet-name>
```

```
  <url-pattern>/hello</url-pattern>
```

```
</servlet-mapping>
```

```
</web-app>
```

4. Explanation of web.xml Tags

- <servlet> → Defines the servlet
- <servlet-name> → Name of the servlet
- <servlet-class> → Full class name of servlet
- <servlet-mapping> → Maps URL to servlet
- <url-pattern> → URL used to access servlet

5. Running the Servlet

1. Place files in server (e.g., Tomcat)
2. Start server
3. Open browser and type:

<http://localhost:8080/project-name/hello>